

5-2 List Reversal

Consider the data type List together with the functions Length and Append below:

```
datatype List<T> = Nil | Cons(head: T, tail: List<T>)

function Length<T>(xs: List<T>): nat
{
  match xs
  case Nil => 0
  case Cons(_, tail) => 1 + Length(tail)
}

function Append<T>(xs: List<T>, ys: List<T>): List<T>
  ensures Length(Append(xs, ys)) == Length(xs) + Length(ys)
{
  match xs
  case Nil => ys
  case Cons(x, tail) => Cons(x, Append(tail, ys))
}
```

A useful property of Append, which will be of use later, is associativity:

```
lemma AppendAssoc<T>(xs: List<T>, ys: List<T>, zs: List<T>)
  ensures Append(Append(xs, ys), zs) ==
    Append(xs, Append(ys, zs))
{}
```

Next, we introduce the function Snoc (the reverse of Cons) to add an element at the end of a list:

which is a specialized appending function, adding a new element at the end of a list, in linear time:

```
function Snoc<T>(xs: List<T>, y: T): List<T>
{
  match xs
  case Nil => Cons(y, Nil)
  case Cons(x, tail) => Cons(x, Snoc(tail, y))
}
```

Snoc is a specialized appending function that can be defined as appending a given list to the front of a singleton list with just the element to be added. Indeed, we can automatically prove the lemma:

```
lemma SnocAppend<T>(xs: List<T>, y: T)
  ensures Snoc(xs, y) == Append(xs, Cons(y, Nil))
{}
```

By using Snoc, we can define a function that reverses a list, albeit in quadratic time:

```
function SlowReverse<T>(xs: List<T>): List<T>
{
  match xs
  case Nil => Nil
  case Cons(x, tail) => Snoc(SlowReverse(tail), x)
}
```

We investigate its properties concerning the Length function.

```
lemma LengthSnoc<T>(xs: List<T>, x: T)
  ensures Length(Snoc(xs, x)) == Length(xs) + 1
{}

lemma LengthSlowReverse<T>(xs: List<T>)
  ensures Length(SlowReverse(xs)) == Length(xs)
{
  match xs
  case Nil => // trivial
  case Cons(x, tail) =>
    calc {
      Length(SlowReverse(xs));
    == Length(Snoc(SlowReverse(tail), x));
    == { LengthSnoc(SlowReverse(tail), x); }
      Length(SlowReverse(tail)) + 1;
    == { LengthSlowReverse(tail); } // ind. hyp.
      Length(tail) + 1;
    == Length(Cons(x, tail));
    }
}
```

In contrast to SlowReverse, a linear algorithm for reversing a list can be defined. First, we define the following tail-recursive function:

```

function ReverseAux<T>(xs: List<T>, acc: List<T>): List<T>
{
  match xs
  case Nil => acc
  case Cons(x, tail) => ReverseAux(tail, Cons(x, acc))
}

```

Then we establish the next lemma, stating that ReverseAux computes the reverse of its first parameter appended in front of its second parameter:

```

lemma ReverseAuxSlowCorrect<T>(xs: List<T>, acc: List<T>)
  ensures ReverseAux(xs, acc) == Append(SlowReverse(xs), acc)
{
  match xs
  case Nil => // trivial
  case Cons(x, tail) =>
    calc {
      ReverseAux(Cons(x, tail), acc);
      == ReverseAux(tail, Cons(x, acc)); // def. of ReverseAux
      == { ReverseAuxSlowCorrect(tail, Cons(x, acc)); } // ind. hyp.
      Append(SlowReverse(tail), Cons(x, acc));
    }
    assert ReverseAux(Cons(x, tail), acc) ==
      Append(SlowReverse(tail), Cons(x, acc));
    calc {
      Append(SlowReverse(Cons(x, tail)), acc);
      == Append(Snoc(SlowReverse(tail), x), acc);
      == { SnocAppend(SlowReverse(tail), x);
          assert Snoc(SlowReverse(tail), x) ==
            Append(SlowReverse(tail), Cons(x, Nil)); }
      Append(Append(SlowReverse(tail), Cons(x, Nil)), acc);
      == { AppendAssoc(SlowReverse(tail), Cons(x, Nil), acc); }
      Append(SlowReverse(tail), Append(Cons(x, Nil), acc));
      == Append(SlowReverse(tail), Cons(x, acc));
    }
    assert Append(SlowReverse(tail), Cons(x, acc)) ==
      Append(SlowReverse(Cons(x, tail)), acc);
}

```

Now, we define a new function, Reverse, by calling ReverseAux with its second parameter set to Nil:

```

function Reverse<T>(xs: List<T>): List<T>
{
  ReverseAux(xs, Nil)
}

```

The resulting function reverses the list in linear time! It remains to prove that it is functionally equivalent to `SlowReverse`, as we do with the next lemmas.

```
lemma AppendNil<T>(xs: List<T>)
  ensures Append(xs, Nil) == xs
{}

lemma ReverseCorrect<T>(xs: List<T>)
  ensures Reverse(xs) == SlowReverse(xs)
{
  calc {
    Reverse(xs);
  == ReverseAux(xs, Nil);
  == { ReverseAuxSlowCorrect(xs, Nil); }
    Append(SlowReverse(xs), Nil);
  == { AppendNil(SlowReverse(xs)); }
    SlowReverse(xs);
  }
}
```

Es. 6.21: Prove the following lemma with the `{:induction false}` clause

```
lemma {:induction false}
ReverseAuxAppend<T>(xs: List<T>, ys: List<T>, acc: List<T>)
  ensures ReverseAux(Append(xs, ys), acc)
    == Append(Reverse(ys), ReverseAux(xs, acc))
{}
```

Ex. 6.22: Prove the following lemma

```
lemma ReverseAppend<T>(xs: List<T>, ys: List<T>)
  ensures Reverse(Append(xs, ys))
    == Append(Reverse(ys), Reverse(xs))
{}
```

Ex. 6.23: Prove that `Reverse` is an involution by proving the lemma

```
lemma ReverseUnit<T>(x: T)
  ensures Reverse(Cons(x, Nil)) == Cons(x, Nil)
{}
```

and then use it in the proof of

```
lemma ReverseReverse<T>(xs: List<T>)
  ensures Reverse(Reverse(xs)) == xs
{}
```

References: Leino, Chapter 6, section 6.